LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: J Satya Srikar
Roll No.: A24126552259

---

1. TITLE

Write a program using JavaScript (arrays, functions)

---

2. AIM

To study and implement JavaScript arrays and functions by performing basic array operations, utilizing built-in array methods, creating multiple types of functions (declarations, expressions, arrow functions, implicit returns), and handling asynchronous operations using async/await.

---

3. OBJECTIVE

The objectives of this experiment are:

- To perform basic array operations including push, pop, shift, and unshift.
- To implement array data transformation and filtering using map(), filter(), and reduce() methods.
- To perform search and retrieval operations using find(), findIndex(), includes(), and indexOf().
- To study advanced patterns including array flattening, ES6 spread operator, and object destructuring.
- To implement multiple function scopes: normal declarations, function expressions, and arrow functions.
- To write asynchronous code blocks using async/await and fetch APIs to simulate server queries.

---

4. THEORY

JavaScript is a versatile, multi-paradigm language that supports first-class functions and dynamic array operations. Arrays in JavaScript are object types capable of storing ordered sequences of elements of varying types. Functions are the fundamental building blocks of JavaScript, behaving as first-class citizens (meaning they can be assigned to variables, passed as arguments, and returned from other functions). Modern JavaScript (ES6+) introduces arrow syntaxes, destructuring, spread operations, and async/await constructs to manage execution loops efficiently.

Theory of JavaScript Arrays & Functions:

- **Array Operations:** Elements are modified using *push()* (append), *pop()* (remove last), *shift()* (remove first), and *unshift()* (prepend). Slicing (*slice()*) copies array parts, while splicing (*splice()*) mutates arrays by inserting/deleting items. Nested sequences are flattened using *flat()*.
- **Functional Iterators:** Functional programming concepts are supported via *map()* (transforms each element), *filter()* (retains elements matching conditions), and *reduce()* (accumulates elements into a single value). Search lookups are handled via *find()* and *includes()*.
- **Spread & Destructuring:** The spread operator (*...*) unpacks elements of arrays or objects, allowing easy concatenation. Destructuring (*const {x, y} = obj*) unpacks values into distinct variables.
- **Function Paradigms:** Traditional function declarations support hoisting. Function expressions bind functions to constants. Arrow functions (*() => {}*) provide clean syntaxes and lexical *this* binding, supporting implicit returns for single-line statements.
- **Asynchronous Functions:** Functions prepended with *async* implicitly return promises. The *await* keyword pauses the function block execution until a promise resolves (e.g. *fetch()* request), leaving the event loop non-blocked.

---

5. ALGORITHM / PROCEDURE

1. Set up Week 3 workspace: create directories *Week3/Src*, *Week3/Docs*, and *Week3/Screenshots*.
2. Create *arrays.js* inside the *Src* directory to implement array manipulations.
3. Add array declarations, push/pop/shift/unshift routines, map/filter/reduce pipelines, slice/splice, and ES6+ spread/destructuring.
4. Create *functions.js* in the *Src* directory to implement function models.
5. Define function declarations, expressions, arrow syntaxes, and implicit return single-liners.
6. Implement an asynchronous function *fetchServerData()* utilizing *fetch()* and *async/await* to query JSON placeholders.
7. Execute both scripts using Node.js in the terminal and capture the console outputs.
8. Verify all outcomes: check array modifications, mapped outputs, async data resolution, and conditional parameters.

---

6. PROGRAM

File: arrays.js

arrays.js (Source Code)

```
//Nobe tire - Creation of an array
const arr = [1,2,3,4,5,6]
    //Construction creation
const arr2 = new Array(1,2,3,4,5,6)
console.log(arr)
console.log(arr2)
arr.push(58)
console.log("pushed 58:"+"["+arr+"]")
arr.pop()
console.log("poped:"+"["+arr+"]")
arr.unshift(2)
console.log("unshifted 1:"+"["+arr+"]")
arr.shift()
console.log("shifted 1:"+"["+arr+"]")

//The Holy Trinity (Mid Tier)
//map function in array
const newarr = arr.map(
    (num)=>{return num * 10}
)
console.log("mapped arr"+newarr)
const booleanArr = arr.filter(
    (num)=>{return num%2 === 0}
)
console.log("booleanArr:"+booleanArr)
const total = arr.reduce((val1_starts_With_zero,val2_starts_with_first_element)=>{return})

//Finding Stuff
function greater(num){
    return num>1;
}
console.log("Find Elements greater than 1",arr.find(greater))
console.log("Find index for number greater than 1",arr.findIndex(greater))
console.log("Does 7 is there in arr",arr.includes(7))
console.log("index of 6 "+arr.indexOf(6))

//slicing & Dicing
let sliced_arr = arr.slice(1,3)
console.log("sliced array",sliced_arr)
let spliced_arr = arr.splice(3,3)
console.log("spliced arr ",spliced_arr)
let unsorted_arr = [4,3,5,1]
console.log("Unsorted array:",unsorted_arr)
console.log("sorted array",unsorted_arr.sort())

//God Mode
// Destructuring
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50
};
console.log("Before Destructuring:",person)
let {firstName, lastName} = person;
console.log("After Destructuring - firstName:",firstName)
console.log("After Destructuring - lastName:",lastName)
console.log("Both Names:",{firstName,lastName})

//Spread Operator
const first = [8,6,5,4,6,6]
const second = [1,2,4,5,6,7]
console.log("first and second",first,second)
const combined = [...first,...second]
console.log("Combined:",combined)

//flat
const Fa = [1,[3,4]]
console.log("Before flat:",Fa)
console.log("After Flat:",Fa.flat())
```

File: functions.js

```
// Normal way of function
// You can call it up here!
console.log(greetOG("Bro"));

function greetOG(name) {
  return `Sup, ${name}`;
}
// Function Expression
const greetMillennial = function(name) {
  return `Hello, ${name}`;
};

console.log(greetMillennial("Bro"));
// Arrow Function (The Gen-Z Way)
const greetGenZ = (name) => {
  return `Yo, ${name}`;
};

// THE GIGACHAD ONE-LINER (Implicit Return)
// If it's just one line of code, you can drop the {} and the 'return' keyword.
const greetFast = name => `Yo, ${name} no cap`;

console.log(greetFast("Bro"));

// Built-in Function
// Math.max() is a built-in function.
// You just hand it numbers and it does the math behind the scenes.
const highestScore = Math.max(45, 120, 13);
console.log(highestScore); // Output: 120

// Put 'async' before the arrow to unlock god mode
const fetchServerData = async () => {
  console.log("1. Sending request to server...");

  // The 'await' keyword tells JS to pause THIS specific function
  // until the fetch finishes, while the rest of your app keeps running fine.
  const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
  const data = await response.json();

  console.log("2. Data secured!", data.name);
};

fetchServerData();
console.log("3. Doing other stuff while waiting...");

// Multi-paramterized functions
const trackStormSystem = (windSpeed, pressure, isLandfall) => {
  if (windSpeed > 119 && isLandfall === true) {
    return `Red Alert: Severe cyclone making landfall. Pressure at ${pressure}hPa.`;
  }
  return "System is stable. Keep monitoring.";
};

// When you call it, you MUST pass the data in the exact same order
const status = trackStormSystem(140, 950, true);
console.log(status);
```

7. CODE EXPLANATION

JavaScript Method / Keyword	Explanation
`push() / pop()`	Appends an element to the end of an array / removes and returns the last element.
`shift() / unshift()`	Removes and returns the first element / prepends one or more elements to the start.
`map(callback)`	Transforms elements by executing a callback on each element, returning a new array.
`filter(callback)`	Creates a new array filtering out elements that fail the condition callback.
`slice(start, end)`	Copies a shallow portion of an array into a new object without mutating the original.
`splice(start, deleteCount)`	Mutates an array by removing, replacing, or inserting elements at a specific index.
`[...arr1, ...arr2]`	Spread operator syntax, unpacking elements to perform array concatenation.
`async / await`	Defines an asynchronous thread loop, pausing execution until promises resolve.
`fetch(url)`	Initiates an asynchronous network request to fetch resources from a remote API.

---

8. TEST CASES AND EXECUTION DATA

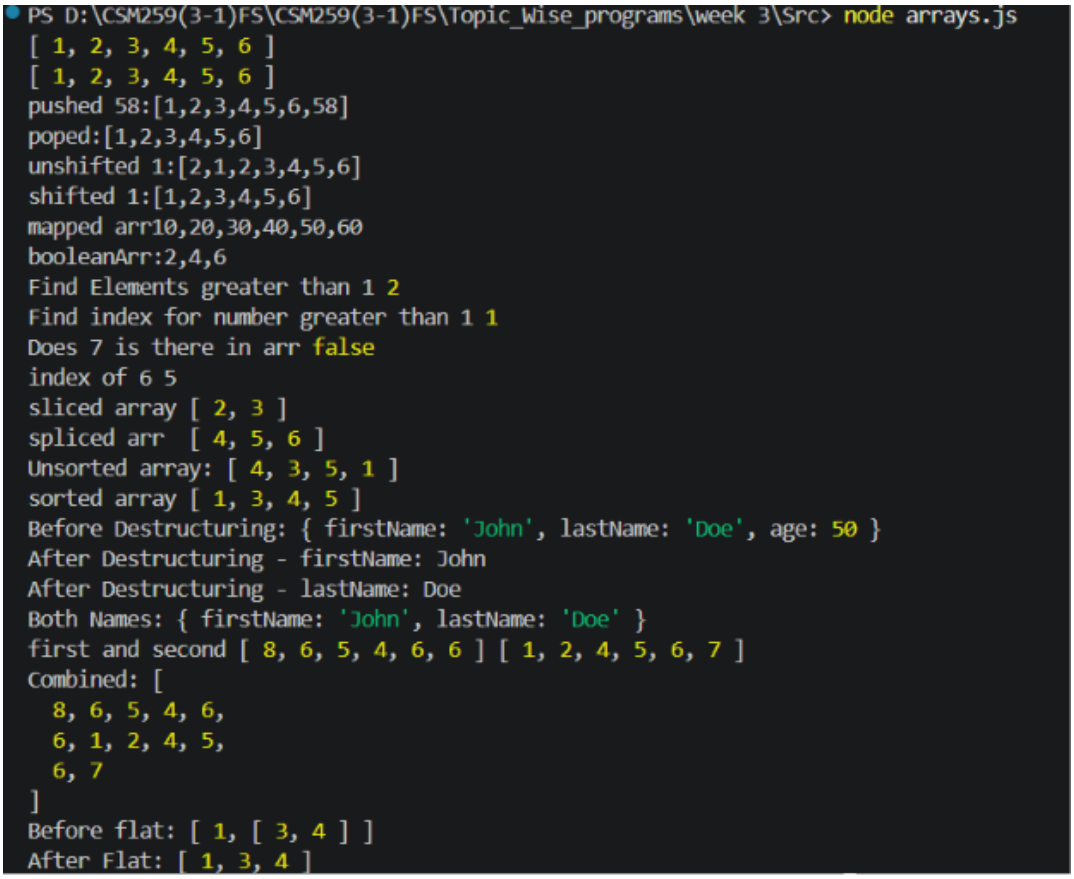
The following test executions verify the execution flow of JavaScript arrays and functions under Node.js:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Array Operations	Run arr.push(), arr.pop()	Array values modify correctly; outputs logged to console	**Pass**
TC-02	Array Iterators	Execute map() and filter()	Transforms (val*10) and filters (even values) return expected arrays	**Pass**
TC-03	Destructuring & Spread	Run spread (...), unpack person	Combined arrays merge; variables capture first/lastName	**Pass**
TC-04	Function Variations	Invoke declarations, expr, arrows	Functions execute, returning greetings based on parameters	**Pass**
TC-05	Asynchronous fetch	Execute fetchServerData()	Queries endpoint asynchronously, printing target name when resolved	**Pass**

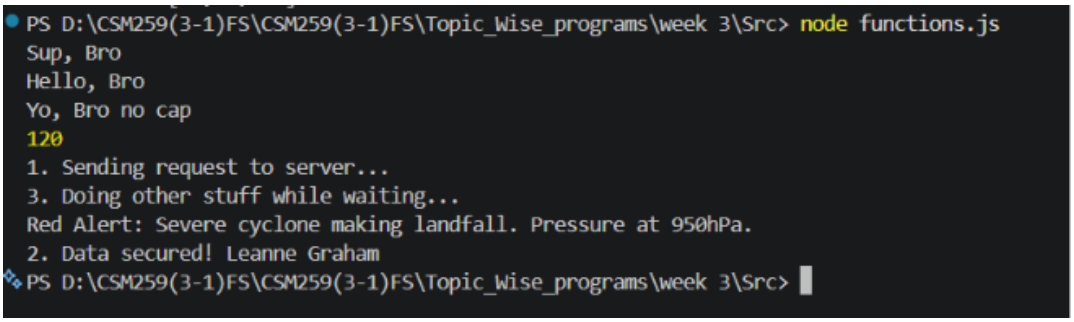
---

9. OUTPUT

Output 1: Execution of arrays.js inside Terminal



Output 2: Execution of functions.js inside Terminal



---

10. RESULT

Thus, JavaScript programs using arrays (operations, filters, transformations, ES6 patterns) and functions (declarations, expressions, arrow styles, async/await routines) were successfully implemented, executed, and verified under Node.js.

Submission Package Structure:

```
Experiment3-Week3/Week3/
■■■ Docs/
■ ■■■ Experiment 3.pdf
■■■ Screenshots/
■ ■■■ arrays.png
■ ■■■ functions.png
■■■ Src/
■■■ arrays.js
■■■ functions.js
```